

Transformée par ondelettes

Et ses applications dans l'analyse d'images

De Haese Tao Piémontèse Enzo Thirion Louis

Cité scolaire Carnot

TIPE 2025

- 1 Introduction à la compression d'images
 - Qu'est ce que compresser une image ?
 - Implémentation naïve
 - Le changement de base
- 2 II. Aspect théorique
 - Algèbre linéaire
 - Les ondelettes de Haar
 - Analyse multi-résolution
- 3 III. Pratique
 - Compression
 - Seuillage
 - Résultats expérimentaux
 - Upscaling par décompression Haar
 - Applications pratiques
 - Résultats
- 4 IV. Conclusion

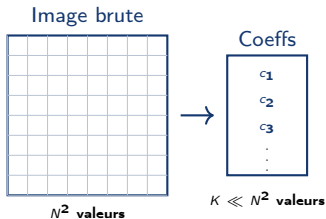
I — Qu'est-ce que compresser une image ?

Une image $N \times N$ en niveaux de gris :
 N^2 entiers dans $\{0, \dots, 255\}$.

Problème :

Stocker les coefficients *importants* ;
supprimer le reste tout en pouvant
reconstruire.

Idée générale : changer de représentation
pour que *peu de nombres* suffisent à
décrire l'image.



Taux de compression

$$\tau = \frac{N^2 - K}{N^2}$$

K = coefficients conservés.

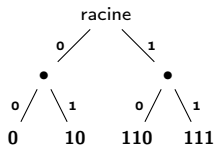
I — Deux stratégies sans transformée

Codage entropique (Huffman)

Les valeurs fréquentes reçoivent un code court.

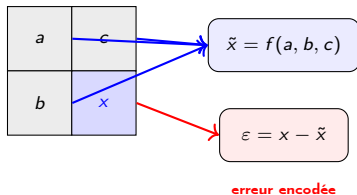
Exemple :

- 1 Pixel gris foncé (fréquent) $\rightarrow 0$
- 2 Pixel tout blanc (rare) $\rightarrow 110101$



Codage prédictif (PNG, lossless)

On prédit chaque pixel à partir de ses voisins et on code seulement l'*erreur de prédiction*.



Limite commune — vers les transformées

Ces méthodes exploitent la *redondance locale*, pas la structure globale. Une **transformation de l'image** concentre l'énergie en peu de coefficients et permet une compression bien plus efficace.

I — L'idée clé : changer de base

Objectif : trouver une représentation dans laquelle *presque tous les coefficients sont nuls* — et ne garder que les importants.

Principe général

Soit (e_k) une base orthonormée. On projette :

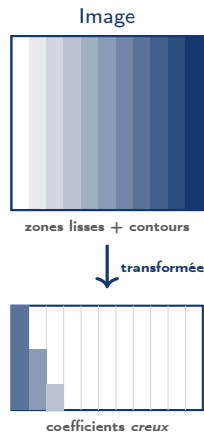
$$c_k = \langle f, e_k \rangle$$

On conserve les K coefficients c_k les plus grands et on met les autres à zéro.

Clé : choisir une base dans laquelle l'image transformée est *creuse*.

Une base idéale capte à la fois :

- La **localisation spatiale**
où est le détail ?
- La **fréquence**
quelle échelle de variation ?



Algèbre linéaire

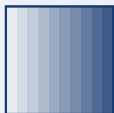
$$L^2(\mathbb{R}) = \left\{ f : \mathbb{R} \rightarrow \mathbb{R} ; \int_{\mathbb{R}} f^2 < +\infty \right\}$$

$$\langle f, g \rangle = \int_{\mathbb{R}} f(x) g(x) dx$$

$$\|f\|^2 = \langle f, f \rangle \triangleq \text{Énergie}$$

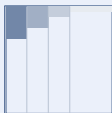
- C'est un espace préhilbertien complet.
- **En dimension finie (discret) :**
on retrouve le *PS* usuel de $\mathbb{R}^N$

Image originale



$$\|f\|^2 = E$$

Coefficients



$$\sum_k c_k^2 = E \quad \text{conservée}$$

Transformée

Importance d'implémentation

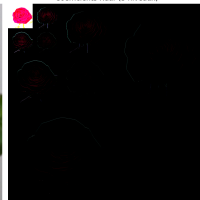
La transformée doit être une **isométrie** $\Leftrightarrow$ conserve l'énergie. Pour l'image cela se traduit par :
Ne pas l'*obscurcir* ou l'*éclaircir* après transformation.

Illustration de l'isométrie

Originale



Coefficients Haar (3 niveaux)



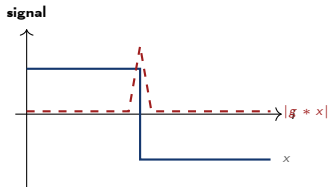
II — Détecter les détails : filtrage et produit scalaire

Un filtre discret h de longueur L appliqué en position i :

Interprétation géométrique

Le filtre **répond fort** quand le signal local *ressemble* à h :

- $h = \frac{1}{\sqrt{2}}(1, 1)$ (passe-bas) :
grande réponse sur zones *uniformes*
- $g = \frac{1}{\sqrt{2}}(1, -1)$ (passe-haut) :
grande réponse sur les *transitions*



Vue matricielle (Haar sur $\mathbb{R}^4$)

$$M = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & -1 \end{pmatrix}$$

Lignes du haut : passe-bas (h)

Lignes du bas : passe-haut (g)

M est orthogonale

$$M^T M = I \quad (\text{filtres orthonormaux})$$

$\Rightarrow M^{-1} = M^T$: reconstruction exacte

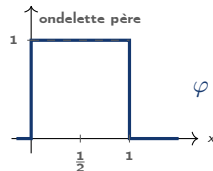
$$\Rightarrow \|Mx\|^2 = \|x\|^2 \quad (\text{Parseval})$$

II — Fonctions de Haar : père et mère

Ondelette père φ

$$\varphi(x) = \begin{cases} 1 & x \in [0, 1[\\ 0 & \text{sinon} \end{cases}$$

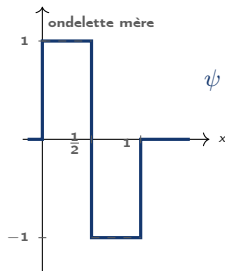
Capture la **tendance** = *moyenne locale*.



Ondelette mère ψ

$$\psi(x) = \begin{cases} +1 & x \in [0, \frac{1}{2}[\\ -1 & x \in [\frac{1}{2}, 1[\\ 0 & \text{sinon} \end{cases}$$

Capture les **détails** = *différence locale*.



Orthonormalité

1 $\langle \varphi, \psi \rangle = 0$

2 $\|\varphi\| = \|\psi\| = 1$

II — Fils et filles : translations et dilatations

À partir de φ et ψ , on génère une famille indexée par l'**échelle** j et la **position** k :

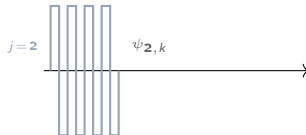
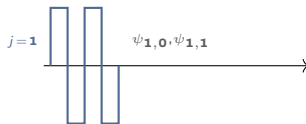
$$\varphi_{j,k}(x) = 2^{j/2} \varphi \left(2^j \left(x - \frac{k}{2^j} \right) \right), \quad \psi_{j,k}(x) = 2^{j/2} \psi \left(2^j \left(x - \frac{k}{2^j} \right) \right), \quad (k, j) \in \mathbb{Z} \times \mathbb{N}$$

- j grand $\Rightarrow$ fenêtre *fine* (HF)
- k décale la position de $k \cdot 2^{-j}$
- $2^{j/2}$ normalise : $\|\psi_{j,k}\| = 1$

Base complète

$\{\varphi_{0,k}\}_{k \in \mathbb{Z}} \cup \{\psi_{j,k}\}_{j \geq 0, k \in \mathbb{Z}}$ est une base orthonormée de $L^2(\mathbb{R})$.

$$f = \sum_k c_k \varphi_{0,k} + \sum_{j \geq 0} \sum_k d_{j,k} \psi_{j,k}$$



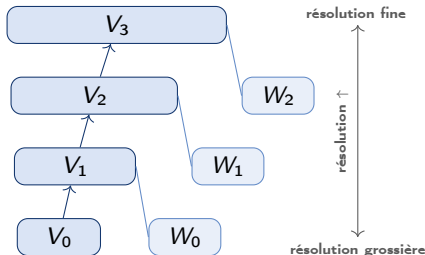
II — Propriétés et analyse multi-résolution (AMR)

Orthogonalité

$$\forall (j, j', k, k') \in \mathbb{N}^2 \times \mathbb{Z}^2 :$$

$$\langle \varphi_{j,k}, \varphi_{j,k'} \rangle = \langle \psi_{j,k}, \psi_{j,k'} \rangle = \delta_{kk'}$$

$$\langle \varphi_{j,k}, \psi_{j',k'} \rangle = 0$$



Espaces emboîtés

On pose $V_j = \text{Vect}\{\varphi_{j,k}\}_{k \in [0, 2^j - 1]}$

$$W_j = \text{Vect}\{\psi_{j,k}\}_{k \in [0, 2^j - 1]}$$

$$\dots \subset V_0 \subset V_1 \subset V_2 \subset \dots \subset L^2(\mathbb{R})$$

$$V_{j+1} = V_j \oplus^\perp W_j$$

$$\text{En itérant : } V_J = V_0 \oplus W_0 \oplus \dots \oplus W_{J-1}$$

Interprétation

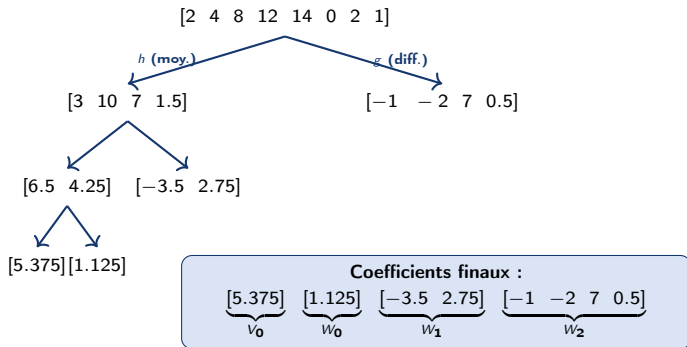
V_j = image lissée à l'échelle 2^{-j}

W_j = détails à l'échelle 2^{-j}

Seuiller $W_j \Leftrightarrow$ supprimer les petits détails

II — Exemple 1D : décomposition de Haar

Signal $N = 8$: [2 4 8 12 14 0 2 1]

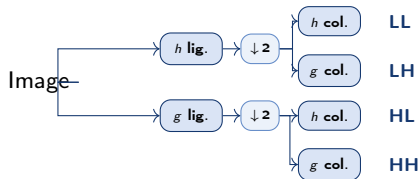


Conclusion sur Haar

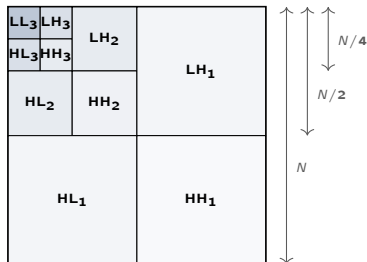
- Les détails (w_j) tendent vers zéro sur les zones lisses $\rightarrow$ idéal pour la compression.
- On peut coder la transformée en **Python** et son inverse

II — Généralisation 2D : les quatre sous-bandes

Principe : filtrer les *lignes* puis les *colonnes*. On peut répéter l'opération sur plusieurs niveaux en repartant de la sous-bande d'approximation.



Sous-bande	Contenu
LL	Approximation (BF)
LH	Contours horizontaux
HL	Contours verticaux
HH	Détails diagonaux



Conséquences

- L'énergie se concentre dans les sous-bandes **LL**. On peut seuiller intelligemment sans trop de *dégâts*
- L'implémentation **Python** 2D se réalise à partir de celle en 1D.

III — Compression : seuillage et énergie

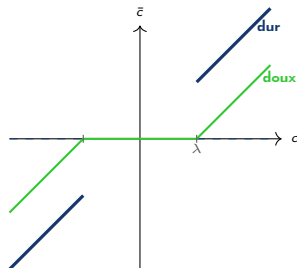
Après la DWT, compresser = annuler les petits coefficients.

Seuillage dur : (Introduit une *discontinuité*)

$$\bar{c} = \begin{cases} c & |c| \geq \lambda \\ 0 & |c| < \lambda \end{cases}$$

Seuillage doux : (Introduit une *perte d'énergie*)

$$\bar{c} = \text{sgn}(c) \max(|c| - \lambda, 0)$$



Erreur contrôlée

$$\|f - \bar{f}\|^2 = \sum_{\bar{c}_k=0} c_k^2$$

- 1 On choisit λ pour contrôler l'erreur quadratique totale.
- 2 Il existe des ondelettes (Cohen–Daubechies–Feauveau), où l'on peut seuiller plus fort.
- 3 On ne seuille pas la sous-bande LL_n (énergie principale).

Notre implémentation utilise le **seuillage dur**.

III — Résultats expérimentaux : taux de compression

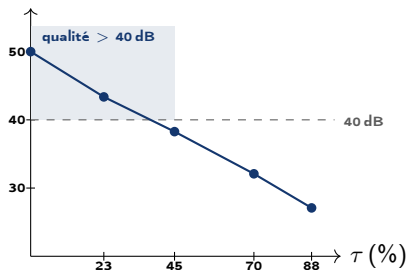
Configuration : image 512×512 , Haar, $L = 3$ niveaux, seuillage dur.

λ	τ (compression)	PSNR
0	0 %	∞
10	23 %	43 dB
20	45 %	38 dB
40	70 %	32 dB
80	88 %	27 dB

$$\text{PSNR} = 10 \log_{10} \frac{255^2}{\text{MSE}}$$

PSNR > 40 dB : qualité visuellement excellente.

PSNR (dB)

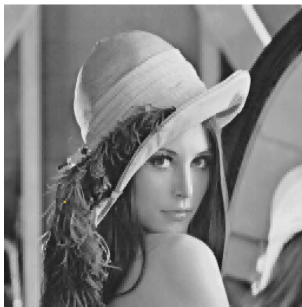


Jusqu'à $\approx 45\%$ de compression avec PSNR > 40 dB : **qualité excellente**.

III — Résultats expérimentaux : Effet du seuillage



$$\lambda = 0$$



$$\lambda = 70$$



$$\lambda = 150$$

Plus λ est élevé, plus un grand nombre de coefficients sont supprimés.

III – Résultats expérimentaux : Comparaison des seuils



- 1 Le seuillage dur laisse transpar tre les discontinuit s dans l'image.
- 2 Le seuillage doux tend   lisser l'image et   l'obscurir l g rement.
- 3 Le seuillage exp est exp rimental et prolonge par continuit  les branches entre le seuillage dur et doux de fa on   *garder le meilleur de chaque*.
- 4 Chaque seuillage peut  tre int ressant pour des images diff rentes.

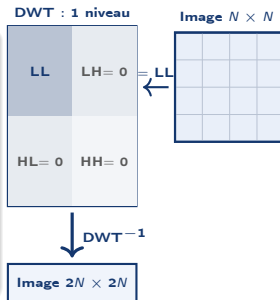
III – Généralisation ok en couleurs

III — Upscaling : agrandir par décompression Haar

Idée : agrandir une image $N \times N$ en $2N \times 2N$ en exploitant la structure de la DWT inverse.

Principe

- 1 Partir de l'image basse résolution ($N \times N$) $\rightarrow$ bloc LL à l'échelle $j = 1$
- 2 **Interpoler** les sous-bandes manquantes LH, HL, HH : on les remplit par zéro (hypothèse : pas de hauts détails)
- 3 Appliquer la **DWT inverse** (Mallat)
 $\rightarrow$ image reconstruite $2N \times 2N$



Lien avec la théorie

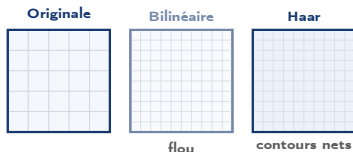
Annuler LH, HL, HH revient à projeter sur V_1 en ignorant W_1 : interpolation dans le sous-espace lisse.

C'est différent d'une interpolation bilinéaire, qui n'exploite pas la structure multi-résolution.

III — Résultats : upscaling Haar vs interpolation

Comparaison sur une image

$128 \times 128 \rightarrow 256 \times 256$:



Méthode	PSNR	Artefacts
Voisin le plus proche	bas	blocs
Bilinéaire	moyen	flou
Haar (DWT^{-1})	bon	flou doux

Pourquoi ça marche

La DWT inverse de Haar redistribue l'énergie de LL sur les 4 quadrants de manière **cohérente avec la structure AMR** :

$$x_{2i,2j} = c + d_h + d_v + d_d$$

$$x_{2i+1,2j} = c - d_h + d_v - d_d$$

$$x_{2i,2j+1} = c + d_h - d_v - d_d$$

$$x_{2i+1,2j+1} = c - d_h - d_v + d_d$$

En posant $d_h = d_v = d_d = 0$, on obtient une **interpolation par répétition locale** — bien plus régulière que le voisin le plus proche.

Ce que les ondelettes apportent

- **Localisation** : $d_{j,k}$ lié à une position *et* une échelle
- **Orthogonalité** : reconstruction exacte, erreur contrôlée via Parseval
- **Efficacité** : $\mathcal{O}(N^2)$ sur une image, inférieur à la FFT 2D en $\mathcal{O}(N^2 \log N)$
- **Moments nuls** : coefficients nuls sur les zones lisses $\Rightarrow$ seuillage efficace

Nos applications

- Compression avec contrôle de la qualité (PSNR)
- Upscaling cohérent avec la structure multi-résolution

Ouverture

JPEG 2000 (CDF 9/7, 4 moments nuls) :

- Pas d'artefacts de blocs
- Compression progressive, scalable
- Standard médical (DICOM)

Aller plus loin :

- *Curvelets / Ridgelets* : bases adaptées aux contours
- *Paquets d'ondelettes* : décomposition adaptative
- *Débruitage* (seuil de Donoho–Johnstone)
- Analyse de turbulence, IRM, watermarking

Annexe — JPEG : la DCT par blocs

Idée : se placer dans une base *fréquentielle* adaptée, puis garder seulement les coefficients importants.

Principe de JPEG

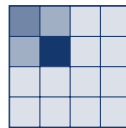
- 1 Découper l'image en blocs 8×8
- 2 Projection des coeffs sur une base de cosinus (adaptée aux fréquences) :

$$C_{p,q} = \lambda_p \lambda_q \sum_{n,m} F[n, m] \cos \frac{\pi p(n+\frac{1}{2})}{8} \cos \frac{\pi q(m+\frac{1}{2})}{8}$$

- 3 Seuillage des HF (invisible à l'oeil nu)
- 4 Coder les coefficients restants (Huffman)



↓
DCT



énergie concentrée
dans les basses fréquences

Défaut de JPEG à fort taux de compression

Ce traitement crée des **artefacts de blocs** : les discontinuités aux frontières deviennent visibles.

Cause : la DCT ne prend pas en compte la continuité entre blocs.

Annexe — Implémentation Python (forward / inverse)

```
def forward(self):  
    res = self.img.copy()  
    n, p = res.shape  
    for _ in range(self.levels):  
        res[:n,:p] = self.t2d(res[:n,:p])  
        n //= 2; p //= 2  
    return res
```

1
2
3
4
5
6

```
def seuillage(coeffs, seuil, mode="dur"):  
    c = coeffs.copy()  
    if mode == "dur":  
        c[np.abs(c) < seuil] = 0.  
    else: # doux  
        s = np.sign(c)  
        c = s * np.maximum(np.abs(c) - seuil,  
                           0.)  
    return c
```

7
8

```
def inverse(self, coeffs):  
    res = coeffs.copy()  
    n = len(res) // 2**(self.levels  
        -1)  
    p = len(res[0]) // 2**(self.levels  
        -1)  
    for _ in range(self.levels):  
        res[:n,:p] = self.t2d_inv(res[:n  
            ,:p])  
        n *= 2; p *= 2  
    return res
```

Annexe — Tableau comparatif JPEG / JPEG 2000 / Haar

	JPEG	JPEG 2000	Haar (notre impl.)
Base	DCT blocs 8×8	DWT CDF 9/7	DWT Haar
Moments nuls	—	4	1
Support filtre	—	9/7	2
Artefacts	blocs	flou doux	flou doux
Scalabilité	non	native	native
Complexité	$\mathcal{O}(N^2)$	$\mathcal{O}(N^2)$	$\mathcal{O}(N^2)$